CS 650 – Full Stack Application Develop

7-1 Activity: Automated Testing and Implementation

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

August 19, 2026

Unit Tests for Back-End Logic

The unit tests focus on the service layer because this is where the application enforces the main business rules for meetups and reservations. The completed test suite directly exercises createMeetup, getMeetup, listMeetups, updateMeetup, deleteMeetup, createReservation, and listReservations using a fresh in-memory SQLite database. Testing these functions independently helps confirm that the back-end logic works correctly before requests are routed through the API or user interface.

The positive tests verify that a meetup can be created with the expected title, description, date, location, and capacity; that a saved meetup returns computed reservationCount and spotsLeft values; that meetups can be listed, updated, and deleted; and that reservations can be created and listed. The test for createReservation also confirms that the returned reservation contains a generated id, the expected name and email, and the correct meetup_id.

Negative and edge cases are included because the most important risk in this application is accepting invalid data or allowing overbooking. The tests verify that a meetup cannot be created with a blank title or a zero or negative capacity. They also verify that a nonexistent meetup returns null, that capacity cannot be reduced below the number of current reservations, that a reservation cannot be created with an empty name, and that an additional reservation is rejected after the meetup reaches capacity. These checks matter because they protect data quality and support the core requirement that reservations never exceed the number of available spots.

```
nikispring@Nikisprings-MacBook-Pro backend % npm install
npm warn deprecated prebuild-install@7.1.3: No longer maintained. Please contact the author of the relevant native addon; alternatives are available.

added 120 packages, and audited 121 packages in 10s

38 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   better-sqlite3@12.11.1 (install: prebuild-install || node-gyp rebuild --release)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.
npm notice
npm notice New major version of npm available! 11.17.0 -> 12.0.2
npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.0.2
npm notice To update run: npm install -g npm@12.0.2
npm notice
nikispring@Nikisprings-MacBook-Pro backend % npm test

> react-meetup-backend@1.0.0 test
> node --test tests/service.test.ts

▶ createMeetup
  ✔ creates a meetup and returns it with correct fields (2.780724ms)
  ✔ throws when capacity is zero or negative (1.6602ms)
  ✔ throws when title is empty (0.257323ms)
✔ createMeetup (11.331309ms)
▶ getMeetup
  ✔ returns meetup with reservationCount and spotsLeft (3.709796ms)
  ✔ returns null for unknown id (0.326245ms)
✔ getMeetup (4.826664ms)
▶ listMeetups
  ✔ returns all meetups with spotsLeft (1.324046ms)
✔ listMeetups (2.600965ms)
▶ updateMeetup
  ✔ updates meetup fields correctly (3.922055ms)
  ✔ throws when capacity is less than current reservations (1.882858ms)
✔ updateMeetup (6.566019ms)
▶ deleteMeetup
  ✔ removes the meetup (1.008801ms)
✔ deleteMeetup (1.569427ms)
▶ createReservation
  ✔ creates a reservation and returns it with correct fields (0.938694ms)
  ✔ throws when meetup is at capacity (1.226901ms)
  ✔ throws when name is empty (0.394673ms)
✔ createReservation (3.378843ms)
▶ listReservations
  ✔ returns all reservations for a meetup (1.32979ms)
✔ listReservations (1.849707ms)
ℹ tests 13
ℹ suites 7
ℹ pass 13
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 447.73103
nikispring@Nikisprings-MacBook-Pro backend %
```

Integration Tests for API Endpoints

The integration tests use Playwright's request API to verify that the Express routes, request validation, service layer, and database work together correctly. Each test checks the HTTP status code and the expected JSON response shape. This is important because a service function can work correctly by itself while the endpoint can still fail because of routing, request parsing, validation, or response handling.

Endpoint	**Method**	**Expected Status**	**Expected Response**
/api/meetups	POST	201	Meetup object with generated id and matching fields
/api/meetups	GET	200	Array of meetup summaries including spotsLeft

/api/meetups/:id	GET	200	Meetup details including reservationCount and spotsLeft
/api/meetups/:id/reservations	POST	201	Reservation object with name, email, and meetup_id
/api/meetups/:id/reservations	GET	200	Array of reservations for the selected meetup
/api/meetups/99999	GET	404	Error response for a nonexistent meetup
/api/meetups	POST invalid body	400	JSON object containing an error message
/api/meetups/:id/reservations when full	POST	409	JSON error indicating the meetup is full

The successful endpoint tests connect directly to the main application requirements: organizers must be able to create and retrieve meetups, community members must be able to reserve a spot, and reservation information must be retrievable. The negative tests cover validation and capacity risks. A 404 response confirms that the application handles an unknown meetup cleanly, a 400 response confirms that malformed meetup data is rejected, and a 409 response confirms that the API communicates a capacity conflict instead of silently overbooking the event.

```
nikispring@Nikisprings-MacBook-Pro e2e % npx playwright test spec/api-integration.spec.ts

Running 7 tests using 1 worker

  ✓  1 …ation Tests › POST /api/meetups creates a meetup and returns 201 (162ms)
  ✓  2 …ntegration Tests › GET /api/meetups lists meetups with spotsLeft (101ms)
  ✓  3 … Integration Tests › GET /api/meetups/:id returns meetup details (109ms)
  ✓  4 …tion Tests › GET /api/meetups/:id returns 404 for non-existent id (81ms)
  ✓  5 …tegration Tests › POST /api/meetups with invalid data returns 400 (92ms)
  ✓  6 …/api/meetups/:id/reservations creates reservation and returns 201 (93ms)
  ✓  7 …Tests › POST /api/meetups/:id/reservations returns 409 when full (105ms)

  7 passed (3.3s)
nikispring@Nikisprings-MacBook-Pro e2e %
```

End-to-End Reservation Workflow

The end-to-end test covers the most important attendee workflow from the browser. The test first creates a meetup through the API so that it begins with a known capacity of 10. It then opens the Register page, finds the correct meetup card, confirms that 10 spots are available, and clicks the Reserve a Spot button. The test enters a name and email address, submits the registration form, and verifies that the confirmation status message is visible and contains the meetup title. Finally, it returns to the meetup list and checks that the card now shows 9 of 10 spots left.

This workflow is important because it crosses every major layer of the application: the React user interface, client-side API calls, Express routing, service logic, and database persistence. A passing result provides stronger evidence than an isolated unit test because it confirms that a real user can complete the primary reservation task and that the visible availability count is updated after the reservation is stored.

```
nikispring@Nikisprings-MacBook-Pro e2e % npx playwright test spec/reservation-flow.spec.ts

Running 2 tests using 1 worker

  ✓  1 … Workflow › create, edit, and delete a meetup via the admin page (975ms)
  ✓  2 …reservation workflow: browse, select, register, see confirmation (604ms)

  2 passed (4.1s)
nikispring@Nikisprings-MacBook-Pro e2e %
```

Nonfunctional Stress Test

The basic stress test is designed to examine how the application behaves when multiple users try to reserve the last available spots at nearly the same time. The test creates a meetup with a capacity of 10 and sends 15 reservation requests concurrently using Promise.all. The expected result is exactly 10 successful responses with status 201 and 5 rejected responses with status 409. After all requests are finished, the test retrieves the meetup and verifies that reservationCount is 10 and spotsLeft is 0.

The parameters are therefore a capacity of 10, 15 total concurrent reservation requests, and five more requests than the available capacity. This test is intended to reveal whether concurrent activity can cause overbooking or an incorrect final availability count. If exactly 10 requests succeed and the final database state still shows 10 reservations with zero spots left, the result indicates that the application maintained the capacity rule under this lightweight burst of concurrent demand.

```
nikispring@Nikisprings-MacBook-Pro e2e % ls spec
api-integration.spec.ts         reservation-flow.spec.ts
helpers.ts                      stress.spec.ts
nikispring@Nikisprings-MacBook-Pro e2e % npx playwright test spec/stress.spec.ts

Running 1 test using 1 worker

  ✓  1 …ns › exactly N reservations succeed for a meetup with capacity N (259ms)

  1 passed (2.8s)
nikispring@Nikisprings-MacBook-Pro e2e %
```

Test Coverage and Results Summary

The test suite connects each test level to a specific application of risk or requirement. Unit tests cover service-level business rules and data validation. Integration tests cover endpoint contracts, status codes, error handling, and communication between the API and database. The end-to-end test covers the complete attendee reservation workflow, while the stress test focuses on the reliability risk of concurrent reservations competing for limited capacity.

Together, the functional tests provide evidence of correctness because they verify the same behavior at several levels. The service tests confirm that the underlying rules work, the API tests confirm that those rules are exposed correctly over HTTP, and the end-to-end test confirms that the user interface can successfully complete the workflow. The stress test adds a nonfunctional perspective by checking whether capacity remains accurate during concurrent requests.

If all tests pass in Codio, the evidence supports release of readiness for the core meetup and reservation features covered by this activity. However, this is not complete with production-level coverage. Additional priorities would include larger and longer load tests, authentication and authorization testing if user accounts are added, security testing, browser and mobile compatibility, accessibility, failure recovery, database locking behavior under heavier traffic, and regression tests for future features. Those areas would provide greater confidence as the meetup platform grows.